
SENTIMENT ANALYSIS SERVICE

Enterprise Technical Assessment Report

Scope: repository reverse engineering, architecture review, system audit, and technical due diligence

Assessment date	2026-06-06
Evidence order	Code, config, infrastructure, tests, legacy docs
Primary runtime	FastAPI + in-process model runtime
Primary UI	Angular single-page application behind Nginx
Primary ML toolchain	DVC, HuggingFace, PEFT/LoRA, MLflow, ONNX Runtime

Tech Lead	Le Quang Tuyen
ML Engineer	Duong Binh An
AI Engineer	Do Quoc Trung
Solution Engineer	Duong Hong Quan
Backend Engineer	Pham Duc Long
UI/UX Engineer	Nguyen Le Hong Nhi

Assessment method

The report is derived from repository inspection. Missing code paths are marked explicitly as Implemented, Partially Implemented, Mocked, Deprecated, or Not Implemented. Claims are limited to implementation evidence present in the repository on 2026-06-06.

Contents

Executive Report	3
1 Section 1 - Executive Technical Summary	3
1.1 System Purpose	3
1.2 Technical Scope	3
1.3 Technology Overview	4
1.4 Core Capabilities	4
1.5 Technical Maturity Assessment	4
1.6 System Classification	5
2 Section 2 - System Architecture Analysis	5
2.1 Architecture Overview	5
2.2 Layer Assessment	5
3 Section 3 - Repository Structure Analysis	6
3.1 Repository Structure Diagram	6
3.2 Major Directory Analysis	7
3.3 Dependency Graph	8
3.4 Module Interaction Diagram	8
4 Section 4 - Execution Flow Analysis	8
4.1 Workflow Matrix	8
4.2 Primary Request Sequence	9
4.3 Explain Sequence	10
4.4 Training to Serving Sequence	10
5 Section 5 - Frontend Analysis	10
5.1 Architecture	10
5.2 Components and Services	11
5.3 Frontend Data Flow	12
5.4 Key Findings	12
5.5 Strengths, Weaknesses, Risks	12
6 Section 6 - Backend Analysis	12
6.1 FastAPI Architecture	12
6.2 API Surface	13
6.3 Contracts and Validation	13
6.4 Middleware and Background Tasks	13
7 Section 7 - AI Inference Analysis	13
7.1 Inference Pipeline	14
7.2 Stage-by-Stage Assessment	14
7.3 Important Inference Findings	14

8 Section 8 - ML Pipeline Analysis	15
8.1 DVC Pipeline Overview	15
8.2 Stage Assessment	15
9 Section 9 - Data Flow Analysis	15
9.1 Data Lineage	16
9.2 Artifact Lifecycle	17
10 Section 10 - API Analysis	18
10.1 Request and Response Summary	18
11 Section 11 - Deployment Analysis	18
11.1 Deployment Diagram	18
11.2 Container and Runtime Notes	19
11.3 Deployment Constraints	19
12 Section 12 - Observability Analysis	19
12.1 Implemented Coverage	20
13 Section 13 - Security Analysis	20
13.1 Security Findings	20
14 Section 14 - Performance Analysis	21
14.1 Performance Profile	21
14.2 Evidence-backed Observations	21
15 Section 15 - Failure Mode Analysis	21
15.1 Failure Tree	21
15.2 Failure Summary	22
16 Section 16 - Technical Debt Analysis	22
17 Section 17 - Scalability Analysis	23
17.1 Scalability View	23
18 Section 18 - Architectural Assessment	23
19 Section 19 - Recommendations	23
20 Section 20 - Final Technical Conclusion	24
20.1 Major Strengths	24
20.2 Major Weaknesses	24
20.3 Critical Risks	25
20.4 Production Readiness and Final Verdict	25

Executive Report

The repository implements a sentiment-analysis product with two tightly coupled planes. The online plane is an Angular frontend served by Nginx, a FastAPI backend in `src/main.py`, and an in-process inference engine centered on `src/model/baseline.py`. The offline plane is an ML artifact pipeline defined by `dvc.yaml`, `params.yaml`, `src/data/`, `src/training/`, and `src/scripts/`. The online plane consumes artifacts produced by the offline plane rather than training models at request time.

The strongest implementation evidence is end-to-end completeness. The repository contains real prediction endpoints, real explainability, synchronous CSV batch processing, DVC-driven data and training stages, ONNX export, Prometheus metrics, Grafana provisioning, MLflow integration, and an automated test suite across contracts, data, model, scripts, and training. The weakest areas are enterprise hardening and operational durability. Authentication is absent. Authorization is absent. `/batch_status/{job_id}` is mocked. Evaluation status is process-memory only. The backend is a single process that owns HTTP serving, model loading, SHAP, ABSA, and batch inference.

The system is best classified as a **hybrid architecture: modular monolith runtime plus embedded ML platform**. It is not a microservice system. The serving layer is one FastAPI deployable. The frontend, Prometheus, Grafana, and MLflow are separate containers, but they do not own domain-specific service boundaries. The ML platform is repository-local and script-driven rather than service-oriented.

Final maturity view

Classification: Hybrid Architecture / Modular Monolith with embedded ML pipeline

Current maturity: Advanced MVP

Production readiness: limited internal production or controlled pilot only

Overall technical verdict: implementation-rich system with clear educational and demo value, but incomplete enterprise controls

1 Section 1 - Executive Technical Summary

1.1 System Purpose

The system converts free-form text into structured sentiment outputs for interactive use and CSV batch use. It also demonstrates the full AI application lifecycle: data download, preprocessing, validation, finetuning, evaluation, ONNX export, runtime loading, prediction, explanation, and monitoring.

1.2 Technical Scope

- frontend user experience in `app/sentiment-analysis-chatbot/`
- backend API runtime in `src/main.py`

- model orchestration in `src/model/`
- contracts in `contracts/`
- data, training, evaluation, and export lifecycle in `src/data/`, `src/training/`, `src/scripts/`, `dvc.yaml`, and `params.yaml`
- deployment and observability manifests in `Dockerfile`, `Dockerfile.train`, `docker-compose.yaml`, and `infra/`

1.3 Technology Overview

Layer	Technology evidence
Frontend	Angular 17 standalone components, signals, RxJS, Lucide icons, Nginx frontend container
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic models from <code>contracts/schemas.py</code>
Inference	Transformers, PEFT, PyTorch, ONNX Runtime, SHAP, zero-shot ABSA
ML lifecycle	DVC, pandas, datasets, scikit-learn, HuggingFace Trainer, MLflow
Observability	Prometheus client, scrape config in <code>infra/prometheus/prometheus.yaml</code> , Grafana provisioning
Packaging	Docker multi-stage runtime image, dedicated training image, Docker Compose topology

1.4 Core Capability Status

Capability	Status	Implementation evidence
Single prediction	Implemented	POST /predict in src/main.py; frontend uses SentimentAnalysisService.predict()
Explainability	Implemented	POST /explain and BaselineModelInference.get_shap_explanation()
CSV batch prediction	Implemented	POST /batch_predict parses upload, caps to 500 rows, dispatches batch inference via asyncio.to_thread()
Durable batch jobs	Mocked	GET /batch_status/{job_id} returns a hardcoded completed payload
Background evaluation trigger	Partially Implemented	/evaluate starts a background task, but status lives in process memory only
Training pipeline	Implemented	DVC stages plus src/scripts/run_finetuning.py and src/training/
ONNX export	Implemented	src/scripts/export_onnx.py and src/model/onnx_exporter.py
Monitoring	Partially Implemented	Prometheus metrics and Grafana provisioning exist; tracing and structured logs do not
Authentication / authorization	Not Implemented	no auth middleware, no dependency, no identity provider, no route protection

1.5 Technical Maturity Assessment

The repository exceeds a toy prototype because it includes a functioning UI, an explicit API contract layer, a concrete inference abstraction, training and export automation, test coverage, and infrastructure manifests. It does not reach enterprise production maturity because trust boundaries, persistent job state, secure secret handling, and workload isolation are missing.

1.6 System Classification

Classification: Hybrid Architecture.

Justification:

- the request-serving backend is one FastAPI monolith with one global model instance
- the frontend is a separate Angular/Nginx container, but not an independently versioned domain service
- monitoring services are support services rather than business services
- the repository embeds a full ML artifact pipeline with DVC and MLflow

2 Section 2 - System Architecture Analysis

2.1 Context Diagram

```

1 flowchart LR
2   User[Browser User] --> FE[Angular Frontend]
3   FE --> NX[Nginx Proxy]
4   NX --> API[FastAPI Runtime]
5   API --> INF[BaselineModelInference]
6   INF --> ONNX[ONNX Runtime Session]
7   INF --> HF[Transformers and PEFT Path]
8   INF --> ABSA[Zero-shot ABSA Pipeline]
9   INF --> SHAP[SHAP Explainer]
10  API --> METRICS[/metrics]
11  METRICS --> PROM[Prometheus]
12  PROM --> GRAF[Grafana]
13  API --> EVAL[/evaluate]
14  EVAL --> MLFLOW[MLflow]
15  DVC[DVC Pipeline] --> MODELS[models/onnx and adapters]
16  MODELS --> INF

```

2.2 Trust Boundary Diagram

```

1 flowchart LR
2   subgraph Browser
3     U[User]
4     LS[localStorage chat history]
5   end
6   subgraph Frontend Container
7     FE[Angular SPA]
8     NX[Nginx]
9   end
10  subgraph Backend Container
11    API[FastAPI]
12    MODEL[Global model instance]
13    MEM[In-memory evaluation state]
14  end
15  subgraph Support Containers
16    PROM[Prometheus]
17    GRAF[Grafana]
18    MLFLOW[MLflow]
19  end
20  subgraph Host Mounted Artifacts
21    DATA[data/]
22    MOD[models/]
23    REP[reports/]
24  end
25  U --> FE
26  FE --> NX
27  NX --> API
28  FE --> LS
29  API --> MODEL

```

```

30  API --> MEM
31  API --> PROM
32  API --> MLFLOW
33  MODEL --> MOD
34  API --> DATA
35  API --> REP

```

2.3 Layer Assessment

Layer	Purpose	Primary files	Inputs	Outputs	Failure modes	Scalability constraints
Frontend	user interaction, health polling, chat state, batch modal, CSV export	<code>app.component.ts</code> , <code>sentiment-analysis-ui/src/app/contracts</code> , <code>batch/message</code> , <code>components</code>	text input, backend health	HTTP requests, rendered messages, localStorage state	backend unavailable, stale local state, contract drift	browser memory and single-tab local state
Backend	routing, schema validation, exception mapping, metrics, evaluation kickoff	<code>src/main.py</code> , <code>contracts/schemas.py</code>	HTTP requests, upload bytes	JSON, text/plain errors, Prometheus format	startup model load failure, CSV parse failure, runtime model exceptions	one process owns heavy synchronous work
Inference	language guard, sentiment, sarcasm, ABSA, SHAP, batch chunking	<code>baseline.py</code> , <code>onnx_inference.py</code> , <code>config.py</code>	text plus language	predictions, explanations	missing artifacts, unsupported language, ABSA/SHAP runtime failure	ABSA and SHAP are CPU-heavy and not cached
Training	artifact creation, finetuning, metrics, benchmark generation	<code>run_finetuning.py</code> , <code>evaluate_finetuned.py</code> , <code>benchmark_onnx.py</code>	raw/processed data, params	adapters, metrics, JSON, benchmark reports	bad raw data, config drift, export mismatch	file-based and container-local execution only
Data	raw data ingestion, preprocessing, validation, evaluation set prep	<code>src/data/</code> , <code>prepare_eval.py</code> , <code>dvc.yaml</code>	XML files, downloaded CSVs, params	processed CSVs, quality report, eval sets	missing external data, schema violations, quality failure	manual artifact placement and file churn
Monitoring	request counting, latency exposure, scrape and dashboard config	<code>src/monitoring/metrics.py</code> , <code>infra/prometheus/fic</code> , <code>infra/grafana/</code>	HTTP traf-fic, scrape calls	counters, histograms, dashboards	scrape alert drift, low metric granularity	no tracing and no app log aggregation

Layer	Purpose	Primary files	Inputs	Outputs	Failure modes	Scalability constraints
Deployment	container build, artifact pull, runtime env wiring	Dockerfile, Dockerfile.train, docker-compose.yml	source code, secrets, DVC credentials	runnable services	DVC pull failure, secret leakage, artifact mismatch	Compose topology only, no rolling update strategy
Security	boundary control, CORS, secret handling, artifact access	runtime config and default network behavior	public HTTP and build credentials	policy outcome or exposed surface	auth absent, wildcard CORS, admin default password	weak boundary model by design

3 Section 3 - Repository Structure Analysis

3.1 Repository Structure Diagram

```

1 flowchart TD
2   ROOT[Repository Root]
3   ROOT --> APP[app/]
4   ROOT --> CONTRACTS[contracts/]
5   ROOT --> DATA[data/]
6   ROOT --> DOCS[docs/]
7   ROOT --> INFRA[infra/]
8   ROOT --> MODELS[models/]
9   ROOT --> SRC[src/]
10  ROOT --> TESTS[tests/]
11  ROOT --> REPORTS[reports/]
12  ROOT --> MLRUNS[mlruns/]
13  ROOT --> DVC[dvc.yaml and params.yaml]
14  ROOT --> DOCKER[Dockerfile and docker-compose.yml]

```

3.2 Critical Path and Runtime Relevance

Directory	Criticality	Runtime relevance	Assessment
src/model/	Critical	Direct runtime path	most important backend module set; owns model mode selection, ONNX/HF loading, ABSA, SHAP, and batch behavior
src/main.py	Critical	Direct runtime path	public API surface and startup lifecycle; breaking changes here are externally visible immediately
contracts/	High	Direct runtime path	shared boundary between HTTP layer, tests, and model abstraction; contract drift here is high-risk
app/	High	Direct runtime path	defines actual end-user capability set, health gating, batch UX, and local state behavior
models/	Critical	Direct runtime dependency	serving depends

3.3 Directory Assessment

Directory	Purpose	Criticality	Dependencies	Modification risk	Complexity	Status
app/	Angular UI and Nginx frontend build	High	backend API and browser APIs	High	Medium	Implemented
contracts/	shared schemas, errors, interface, mock model	High	FastAPI, tests	High	Low	Implemented
data/	raw, processed, eval, and quality artifacts	High	DVC stages and scripts	High	Medium	Implemented
infra/	Prometheus scrape rules and Grafana provisioning	Medium	Compose network and metrics endpoint	Medium	Low	Implemented
models/	LoRA adapters and ONNX artifacts	Critical	training/export scripts and runtime loader	Critical	Medium	Implemented
src/data/	dataset download, preprocess, validate	High	params and raw data	High	Medium	Implemented
src/model/	inference, evaluation, ONNX runtime and export support	Critical	ML libraries and artifacts	Critical	High	Implemented
src/monitoring/	request counters and latency histograms	Medium	FastAPI middleware	Medium	Low	Partially Implemented
src/scripts/	CLI lifecycle tasks	High	training, data, model modules	High	Medium	Implemented
src/training/	trainer, dataset, metrics, task config helpers	High	transformers and sklearn stack	High	Medium	Implemented
tests/	API, contracts, model, data, scripts, training coverage	Medium	pytest and mocks	Medium	Medium	Implemented

3.4 Dependency Graph

```

1 flowchart LR
2   APP --> CONTRACTS
3   APP --> API[src/main.py]
```

```

4  API --> CONTRACTS
5  API --> MODEL[src/model]
6  API --> MON[src/monitoring]
7  MODEL --> CFG[src/model/config.py]
8  MODEL --> ONNX[src/model/onnx_inference.py]
9  SCRIPTS[src/scripts] --> TRAIN[src/training]
10 SCRIPTS --> DATA[src/data]
11 SCRIPTS --> MODEL
12 DVC[dvc.yaml] --> SCRIPTS
13 DVC --> DATA
14 DVC --> TRAIN
15 MODEL --> ART[models/]

```

3.5 Module Interaction Diagram

```

1  flowchart LR
2    AppComponent --> SentimentService
3    SentimentService --> PredictAPI[/predict]
4    SentimentService --> ExplainAPI[/explain]
5    SentimentService --> BatchAPI[/batch_predict]
6    SentimentService --> HealthAPI[/health]
7    PredictAPI --> BaselineModelInference
8    ExplainAPI --> BaselineModelInference
9    BatchAPI --> BaselineModelInference
10   BaselineModelInference --> OnnxInferenceSession
11   BaselineModelInference --> HFModel
12   BaselineModelInference --> ABSA
13   BaselineModelInference --> SHAP

```

4 Section 4 - Execution Flow Analysis

4.1 Workflow Matrix

Workflow	Entry point	Execution path	Key files	Outputs	Status
Startup	FastAPI lifespan	env → ModelConfig → BaselineModelInference → model load → ABSA preload	src/main.py, src/model/baseline.py, src/model/config.py	global loaded or startup abort	Implemented
Health check	GET /health	dependency injection → get_model() → static payload build	src/main.py, contracts/schemas.py	HealthResponse	Implemented
Prediction	POST /predict	validate → resolve language → predict → metrics → shape response	src/main.py, baseline.py, language_detector.py	PredictResponse	Implemented
Explanation	POST /explain	validate → resolve language → SHAP explanation → response	src/main.py, baseline.py	ExplainResponse	Implemented

Workflow	Entry point	Execution path	Key files	Outputs	Status
Batch processing	POST /batch_predict	read upload → parse CSV → cap 500 → filter empty → threaded batch inference → row rebuild	src/main.py, baseline.py	BatchPredictResponse	Implemented
Batch status	GET /batch_status/{id}	return hardcoded command state	src/main.py	fixed status payload	Mocked
Evaluation	POST /evaluate	background task → load params → load processed data → evaluate → log MLflow	src/main.py, src/model/evaluate.py	JSON status + MLflow run	Partially Implemented
Training	CLI / DVC	download/prep → fine-tune adapter → save outputs	dvc.yaml, run_finetuning.py, src/training/	adapters and local MLflow runs	Implemented
ONNX export	CLI / DVC	load adapter → export FP32 → quantize INT8	export_onnx.py, onnx_exporter.py	ONNX directories	Implemented
Deployment	docker compose up	build images → start support services → backend loads artifacts → frontend proxies	Dockerfiles and Compose	multi-container local stack	Implemented

4.2 Startup Sequence

```

1 sequenceDiagram
2   participant Uvicorn
3   participant FastAPI
4   participant Lifespan
5   participant Config as ModelConfig
6   participant Model as BaselineModelInference
7   participant Artifacts as models/
8   Uvicorn->>FastAPI: start app
9   FastAPI->>Lifespan: enter lifespan()
10  Lifespan->>Config: read MODEL_MODE and build config
11  Lifespan->>Model: instantiate global model
12  Model->>Artifacts: load ONNX or HF artifacts
13  Model->>Model: preload ABSA pipeline
14  alt load success
15      Lifespan-->>FastAPI: app ready
16  else load failure
17      Model-->>Lifespan: raise ModelError
18      Lifespan-->>FastAPI: startup abort
19  end

```

4.3 Prediction Sequence

```

1 sequenceDiagram

```

```

2 participant User
3 participant FE as Angular Service
4 participant API as FastAPI /predict
5 participant Lang as LanguageDetector
6 participant Model as BaselineModelInference
7 participant Metrics as Prometheus Histograms
8 User->>FE: submit text
9 FE->>API: POST /predict
10 API->>Lang: resolve_request_language()
11 API->>Model: predict_single(text, lang)
12 Model->>Model: _predict_probabilities()
13 Model->>Model: _extract_aspects()
14 Model->>Model: _predict_sarcasm_flag()
15 API->>Metrics: observe inference latency
16 API-->>FE: PredictResponse
17 FE-->>User: render sentiment, aspects, sarcasm, latency

```

4.4 Batch Sequence

```

1 sequenceDiagram
2 participant User
3 participant FE as BatchUploadComponent
4 participant API as FastAPI /batch_predict
5 participant Pandas
6 participant Worker as asyncio.to_thread
7 participant Model as BaselineModelInference
8 User->>FE: upload CSV
9 FE->>API: multipart POST /batch_predict
10 API->>Pandas: read_csv(bytes)
11 API->>API: validate text column and cap 500 rows
12 API->>Worker: run blocking batch inference
13 Worker->>Model: predict_batch(valid_texts, skip_absa=True)
14 Model-->>Worker: list of PredictionResult
15 API->>API: rebuild per-row results and errors
16 API-->>FE: BatchPredictResponse
17 FE-->>User: render summary and exportable results

```

4.5 Evaluation Sequence

```

1 sequenceDiagram
2 participant Caller
3 participant API as FastAPI /evaluate
4 participant Task as BackgroundTask
5 participant Params as params.yaml
6 participant Data as data/processed/sentences.csv
7 participant Eval as src/model/evaluate.py
8 participant MLflow
9 Caller->>API: POST /evaluate
10 API->>API: check _evaluate_state.running
11 API->>Task: add _do_evaluate
12 API-->>Caller: started
13 Task->>Params: load params

```

```

14 Task->>Data: read processed data
15 Task->>Eval: evaluate_on_dataset()
16 Task->>MLflow: log metrics
17 Task->>API: update in-memory status

```

5 Section 5 - Frontend Analysis

5.1 Architecture

The frontend is an Angular standalone-component application. `AppComponent` is the shell. `SentimentAnalysisService` owns API calls and chat history state. Batch processing is isolated in `BatchUploadComponent`. Explanation interaction is coordinated by `MessageBubbleComponent`, while health polling happens in `AppComponent.ngOnInit()`.

5.2 Component and State Model

Component / service	Role	Evidence-backed behavior
<code>AppComponent</code>	application shell	polls <code>/api/health</code> every 3 seconds until success, controls modal visibility, theme, mobile sidebar, and loading overlay
<code>SentimentAnalysisService</code>	state and API service	stores messages in an Angular signal, persists chat history to <code>localStorage</code> , calls <code>/api/predict</code> , <code>/api/explain</code> , <code>/api/batch_predict</code> , and <code>/api/health</code>
<code>BatchUploadComponent</code>	batch workflow UI	validates CSV extension on selection, sends multipart request, renders summary cards and detailed table, exports client-side CSV
<code>MessageBubbleComponent</code>	result rendering	toggles explanation fetch for prior bot messages and displays returned SHAP payload
<code>ThemeService</code>	UI preference	toggles dark mode state locally in frontend only

5.3 Frontend Data Flow

```

1 flowchart LR
2   Input[ChatInput or BatchUpload] --> Service[SentimentAnalysisService]
3   Service --> Health[/api/health]
4   Service --> Predict[/api/predict]
5   Service --> Explain[/api/explain]
6   Service --> Batch[/api/batch_predict]
7   Service --> Signal[message signal]
8   Signal --> UI[AppComponent and child components]
9   Signal --> Storage[localStorage]

```

5.4 API Integration

- `checkHealth()` calls `/api/health` with no local schema wrapper.
- `predict()` posts only `{text}`; the current UI does not send `lang` even though the backend schema supports it.
- `callExplainApi()` posts only `{text}` and attaches results to the existing bot message.
- `batchPredict()` submits a multipart `FormData` payload containing one file field named `file`.

5.5 UI Interaction Flows

- startup flow: render loading overlay, poll health, then inject a welcome message
- single prediction flow: append user message, append loading bot message, replace loading state when response arrives
- explainability flow: find the nearest preceding user message, call `/api/explain`, attach SHAP output to the existing bot message
- batch flow: open modal, select CSV, run batch, render summary cards and export results CSV locally

5.6 Strengths, Weaknesses, and Risks

Dimension	Assessment
Strengths	clean service boundary, browser-only persistence is simple, batch UX is concrete, health-gated startup improves user feedback
Weaknesses	no server-persisted session state, no retry policy beyond basic error messages, health contract typed as <code>any</code> , no upload progress indicator
Risks	<code>localStorage</code> can become stale or malformed, frontend readiness can be optimistic when chat history exists, no auth-aware UX because auth does not exist
Maintainability concerns	large inline templates in components and business logic mixed with presentation increase change risk

6 Section 6 - Backend Analysis

6.1 FastAPI Architecture

The backend is a single FastAPI application with a global model singleton and request middleware. The app lifecycle is startup-heavy because model loading and ABSA preload happen in the lifespan function. There is no service layer split separate from route handlers; routing, response shaping, and metrics observation remain in `src/main.py`.

6.2 Routing and Dependency Injection

- dependency: `get_model()` returns the global `ml_model` or raises HTTP 503
- helper: `resolve_request_language()` returns explicit `lang` if provided, else uses `LanguageDetector`
- middleware: CORS plus Prometheus middleware from `src/monitoring/metrics.py`
- exception handlers: `UnsupportedLanguageError` mapped to HTTP 400 and `ModelError` mapped to HTTP 500 as `text/plain`

6.3 Validation and Contracts

Validation is primarily schema-driven in `contracts/schemas.py`. `PredictRequest` enforces text length 1 to 2000. `ExplainResponse` validates that `tokens` and `shap_values` lengths match. Batch CSV validation is route-local and checks parseability and presence of a `text` column.

6.4 Backend Control Flow

```
1 flowchart LR
2   Request --> CORS
3   CORS --> Monitor[monitor_middleware]
4   Monitor --> Route[src/main.py routes]
5   Route --> Dependency[get_model()]
6   Route --> Lang[resolve_request_language]
7   Route --> Model[BaselineModelInference]
8   Route --> Schema[Pydantic response models]
9   Schema --> Response
```

6.5 Background Tasks and Long-running Work

- batch inference is not a FastAPI background task; it is executed with `asyncio.to_thread()` inside the request scope
- evaluation uses `BackgroundTasks.add_task()` and stores state in module-global `_evaluate_state`
- there is no queue, no worker process, and no durable state store

6.6 Metrics Collection

Middleware records request count and request latency for all paths. The prediction route separately observes model inference latency by normalized language label. There is no metric for batch row count, SHAP latency, ABSA latency, startup duration, or evaluation duration.

7 Section 7 - AI Inference Analysis

7.1 Model Loading Strategy

Mode	Loader path	Behavior
onnx / onnx_int8	BaselineModelInference._load_model() → OnnxInferenceSession	loads senti- ment ONNX ses- sion and at- tempts to load match- ing sar- casm ONNX ses- sion by path sub- stitu- tion
finetuned	tokenizer + base HF model + PEFT sentiment and sarcasm adapters	keeps one model object and switches adapters for senti- ment ver- sus sar- casm pre- dic- tion
baseline	tokenizer + base HuggingFace model	no fine- tuned adapter, no sar- casm model, no ONNX

7.2 Inference Pipeline

```

1 flowchart LR
2   Text --> LangGuard[_check_language]
3   LangGuard --> Sentiment[_predict_probabilities]
4   Sentiment --> SentimentLabel[label_map argmax]
5   SentimentLabel --> ABSA[_extract_aspects]
6   SentimentLabel --> Sarcasm[_predict_sarcasm_flag]
7   SentimentLabel --> Response[PredictionResult]
8   Text --> SHAP[get_shap_explanation]

```

7.3 Stage Analysis

Stage	Input	Transformation	Output	Cost	pro-	Failure modes
				file		
Language guard	text, optional lang	explicit pass-through or detector call	normalized language code	low CPU		unsupported language later raises 400
Sentiment scoring	text or batch chunk	ONNX or HF forward pass, softmax, argmax mapping	class probabilities and label	primary runtime cost		artifact load errors, runtime inference errors
Sarcasm scoring	one text	ONNX sarcasm session or fine-tuned sarcasm adapter	boolean flag	sequential per text		missing sarcasm model degrades to false
ABSA	one text	zero-shot pipeline over configured aspect categories and threshold	list of expensive AspectSentiment	GPU path		pipeline load failure at startup or per-call warning with empty result
SHAP	one text	build prediction function, build SHAP explainer, compute token contributions	SHAPResult	highest per-request cost		tokenizer/pipeline/SHAP runtime errors
Batch prediction	list of texts	chunked inference, optional ABSA skip and sarcasm skip	list of scales with PredictionResult	with chunk count and per-row post-processing		invalid batch size, runtime inference failure, iterator mismatch risk if logic changes

7.4 Inference Optimization Status

- implemented: ONNX runtime mode, INT8 export path, batch chunking, language-label normalization for metrics, ABSA preload to shift cost to startup
- partially implemented: batch route skips ABSA to control latency, but sarcasm remains per-row unless explicitly disabled

- not implemented: model pool isolation, request-level timeout control, SHAP caching, ABSA caching, concurrency throttling

8 Section 8 - ML Pipeline Analysis

8.1 DVC Pipeline Graph

```

1 flowchart TD
2   download --> preprocess
3   preprocess --> validate
4   preprocess --> evaluate_baseline
5   download_sarcasm --> finetune_sarcasm
6   download_sentiment --> prepare_eval
7   download_sentiment --> finetune_sentiment
8   finetune_sarcasm --> evaluate_fineturned
9   finetune_sentiment --> evaluate_fineturned
10  finetune_sarcasm --> export_onnx_sarcasm
11  finetune_sentiment --> export_onnx_sentiment
12  export_onnx_sentiment --> benchmark_onnx

```

8.2 Pipeline Stage Analysis

Stage	Purpose	Inputs	Outputs / artifacts	Dependencies	Status
download	fetch SemEval raw data	src/data/download_external XML files	data/raw/sentences.csv, data/raw/aspects.csv	DVC, downloader script	Implemented
preprocess	transform raw data into processed data	raw CSVs, transforms, params	data/processed/	src/data/pipeline.py	Implemented
validate	quality checks on processed data	processed data and validation params	data/reports/quality_checks.json	validation module	Implemented
evaluate_baseline	baseline evaluation with MLflow params	processed sentences and model code	data/reports/baseline_metrics.json	evaluation module	Implemented
download_sarcasm / download_sentiment	fetch task-specific training CSVs	downloader script and params	task raw CSVs	downloader module	Implemented
prepare_eval	build evaluation datasets from raw sentiment splits	raw sentiment CSVs and evaluation params	data/eval/	src/scripts/prepare_eval.py	Implemented
finetune_*	produce task adapters	task data, training helpers, params	models/adapters/*run_finetuning.py		Implemented

Stage	Purpose	Inputs	Outputs / artifacts	Dependencies	Status
<code>evaluate_finetuned</code>	evaluate fine-tuned artifacts and fairness	adapters and eval sets	metrics JSON reports in <code>reports/</code>	eval script and model modules	Implemented
<code>export_onnx_*</code>	create serving artifacts	adapters and ONNX exporter	FP32 and INT8 directories	export script	Implemented
<code>benchmark_onnx</code>	benchmark runtime artifact performance	ONNX artifacts and benchmark script	<code>reports/onnx_benchmark.json</code>	benchmark script	Implemented

8.3 ML Lifecycle Observations

- the pipeline is artifact-oriented rather than service-oriented
- DVC is the orchestration source of truth for offline sequencing
- MLflow is used for experiment logging, but there is no model registry promotion workflow in code
- runtime artifacts and training outputs are colocated under `models/`, which simplifies local use but weakens environment isolation

9 Section 9 - Data Flow Analysis

9.1 End-to-End Data Lineage

```

1 flowchart LR
2   Raw[Raw XML and downloaded CSV] --> Prep[Preprocess and validation]
3   Prep --> Processed[data/processed]
4   Processed --> Train[Finetuning]
5   Processed --> EvalBase[Baseline evaluation]
6   Train --> Adapters[models/adapters]
7   Adapters --> Export[ONNX export]
8   Export --> OnnxArtifacts[models/onnx]
9   OnnxArtifacts --> Runtime[BaselineModelInference]
10  Runtime --> Prediction[Predict and Explain APIs]

```

9.2 Artifact Inventory

Artifact	Type	Produced by	Consumed by	Location
SemEval raw extracts	raw tabular data	download stage	preprocess	<code>data/raw/sentences.csv</code> , <code>data/raw/aspects.csv</code>
Processed training data	processed dataset	preprocess	validation, baseline evaluation, downstream analysis	<code>data/processed/</code>

Artifact	Type	Produced by	Consumed by	Location
Quality report	metrics JSON	<code>validate</code>	human review / CI evidence	<code>data/reports/quality_report.json</code>
Task raw datasets	raw CSVs	task-specific download stages	finetuning, eval prep	<code>data/raw/sarcasm.csv</code> , <code>data/raw/sentiment_*.csv</code>
Evaluation sets	evaluation CSVs	<code>prepare_eval</code>	finetuned evaluation	<code>data/eval/</code>
LoRA adapters	model artifacts	<code>finetune_*</code>	ONNX export, finetuned runtime mode	<code>models/adapters/</code>
ONNX sentiment / sarcasm models	runtime model artifacts	<code>export_onnx_*</code>	ONNX runtime serving	<code>models/onnx/</code>
Benchmark and metrics reports	JSON reports	eval and benchmark scripts	human review / docs	<code>reports/</code> , <code>data/reports/</code>

9.3 Transformation Logic

- preprocessing transforms external raw data into normalized sentence-level and aspect-level forms
- validation enforces data quality and writes a non-cached report
- finetuning converts labeled task datasets into adapters
- export converts adapters into ONNX artifacts for low-latency serving
- runtime never reads training code directly; it reads exported artifacts and static config

10 Section 10 - API Analysis

10.1 Endpoint Inventory

Method	Path	Purpose	Request / response	Key validation	Auth
GET	<code>/health</code>	expose model readiness and supported languages	no request; <code>HealthResponse</code>	model dependency must exist	None
POST	<code>/predict</code>	single-text prediction	<code>PredictRequest</code> → <code>PredictResponse</code>	text length 1–2000, language support enforced by model	None
POST	<code>/explain</code>	token-level SHAP explanation	<code>ExplainRequest</code> → <code>ExplainResponse</code>	schema plus token/value length equality on response	None

Method	Path	Purpose	Request / response	Key validation	validation	Auth
POST	/batch_predict	synchronous CSV batch prediction	multipart file → CSV BatchPredictResponse	CSV text present, first 500 rows only	parse, column first	None
GET	/batch_status/{job_id}	batch job status	path param → BatchStatusResponse	none meaningful; returns fixed payload	one in-run	None
POST	/evaluate	trigger async evaluation	no formal body; JSON status object	only flight allowed	one in-run	None
GET	/evaluate/status	inspect in-memory evaluation state	no request; generic JSON	none		None
GET	/metrics	expose Prometheus metrics	no request; Prometheus plain-text	none		None

10.2 Request and Failure Behavior

Endpoint	Failure behavior
/health	returns 503 indirectly if <code>get_model()</code> raises because startup failed or is incomplete
/predict	unsupported language maps to 400, generic model errors map to 500
/explain	same model-related exception mapping as /predict; SHAP errors bubble through <code>ModelError</code> if wrapped or generic 500 otherwise
/batch_predict	invalid CSV and missing <code>text</code> column return 400; inference failures return 500
/evaluate	returns 409 if evaluation already running; background task records last error in memory only
/metrics	no dedicated auth or throttling, so scrape endpoint is publicly reachable wherever the API is reachable

11 Section 11 - Deployment Analysis

11.1 Compose Topology

```

1 flowchart LR
2   Browser --> Frontend[frontend:80]
3   Frontend --> Backend[fastapi_app:8000]
4   Backend --> MLflow[mlflow:5000]
5   Backend --> Metrics[/metrics/]
6   Metrics --> Prometheus[prometheus:9090]
7   Prometheus --> Grafana[grafana:3000]
8   Trainer[trainer profile] --> Data[data/]
9   Trainer --> Models[models/]
10  Backend --> Models

```

11.2 Service Inventory

Service	Role	Ports	Notable config
<code>fastapi_app</code>	runtime API and model host	8000:8000	DVC-pulled artifacts in image build, CPU and memory limits, MLflow tracking URI
<code>frontend</code>	Angular UI over Nginx	80:80	depends on backend, proxies API traffic
<code>prometheus</code>	scrape metrics	9091:9090	scrape config and alert rules mounted from <code>infra/prometheus/</code>
<code>grafana</code>	dashboards	3000:3000	admin password set to <code>admin</code> , provisioning mounted from repo
<code>mlflow</code>	experiment tracking server	5005:5000	local server mode, no auth configuration
<code>trainer</code>	optional offline training container	profile only	host-mounted <code>data/</code> , <code>models/</code> , <code>mlruns/</code> , <code>reports/</code>

11.3 Dockerfile Findings

- runtime image installs DVC and pulls selected ONNX artifacts and processed sentences during build
- build requires DagsHub credentials through `DAGSHUB_USERNAME` and `DAGSHUB_TOKEN`
- runtime image intentionally pulls sentiment and sarcasm ONNX artifacts rather than training inside the serving container
- training has a separate `Dockerfile.train` that excludes ONNX runtime and is used only through the Compose `trainer` profile

11.4 Environment Variables and Startup Order

Variable	Consumer	Purpose
<code>MODEL_MODE</code>	FastAPI startup	selects ONNX, ONNX INT8, fine-tuned, or baseline mode
<code>MLFLOW_TRACKING_URI</code>	backend and trainer	selects MLflow destination
<code>OMP_NUM_THREADS</code> , <code>MKL_NUM_THREADS</code>	backend and trainer	adjusts CPU thread usage
<code>DAGSHUB_USERNAME</code> , <code>DAGSHUB_TOKEN</code>	runtime image build	required for DVC artifact pull during image build
<code>GF_SECURITY_ADMIN_PASSWORD</code>	Grafana	sets admin password, currently <code>admin</code> in compose

12 Section 12 - Observability Analysis

12.1 Logging and Metrics

Metrics are implemented. Logging is minimal and mostly implicit through Python logging in model code plus framework defaults. There is no structured JSON logging, no correlation ID, and no log sink configuration in Compose.

12.2 Metrics Inventory

Metric	Type	Evidence-backed meaning
api_requests_total	Counter	counts requests by method, endpoint, and HTTP status in middleware
api_request_latency_seconds	Histogram	measures whole-request latency by method and endpoint
model_inference_latency_seconds	Histogram	records latency for <code>/predict</code> model inference only, labeled by normalized language

12.3 Telemetry Flow

```
1 flowchart LR
2   Request --> Middleware[monitor_middleware]
3   Middleware --> MetricsMem[prometheus_client registry]
4   Prometheus -->|scrape /metrics| API
5   API --> MetricsMem
6   Prometheus --> Grafana
```

12.4 Monitoring Coverage and Gaps

- covered: endpoint volume, endpoint latency, prediction inference latency
- partially covered: alerting files exist, but alert operational quality is not proven by tests here
- not covered: startup duration, model load failures, SHAP latency, ABSA latency, batch row counts, queue depth, memory, CPU, error taxonomy
- not implemented: tracing, log aggregation, SLO reporting, anomaly detection

13 Section 13 - Security Analysis

13.1 Evidence-based Findings

Severity	Finding	Evidence	Impact	Recommendation
Critical	no authentication or authorization on any route	<code>src/main.py</code> exposes all endpoints without auth dependencies	any reachable client can call prediction, explanation, evaluation, and metrics	add API auth at gateway or app layer before external exposure
High	wildcard CORS allows all origins with credentials enabled	<code>CORSMiddleware</code> <code>allow_origins=["*"]</code> and <code>allow_credentials=True</code>	broad browser attack surface and poor boundary hygiene	restrict allowed origins and disable credential support unless required
High	Grafana default admin password is committed in compose	<code>GF_SECURITY_ADMIN_PASSWORD=admin</code> in <code>docker-compose.yml</code>	compromise in composed environments	externalize secret and rotate default admin credentials
High	build-time artifact pull requires secrets in image build context	<code>Dockerfile</code> uses DVC pull with DagsHub credentials	secret mishandling risk in local and CI builds	move to secure build secret mechanism or pre-provision artifacts
Medium	admin-style evaluation trigger is public	<code>POST /evaluate</code> has no access control	untrusted users can force expensive background evaluation	protect the route or remove from serving deployment
Medium	metrics endpoint is public	<code>GET /metrics</code> has no auth	leaks operational metadata to any reachable caller	network-restrict or gateway-protect metrics
Medium	batch upload trusts file extension at frontend and content parse at backend	frontend checks extension; backend parses arbitrary CSV bytes	malformed or oversized uploads can cause resource pressure	add size limits, MIME checks, and server-side quotas
Low	localStorage retains prior chat content	<code>SentimentAnalysisService.saveHistory()</code>	persist on shared browsers	explicit privacy notice or opt-out setting

14 Section 14 - Performance Analysis

14.1 Observed Architecture Constraints

The repository includes a benchmark stage for ONNX but the report cannot claim general production timings without executing the workloads. What is implementation-evident is the performance shape:

- startup is expensive because the app loads the primary model and preloads the ABSA pipeline in lifespan
- `/predict` performs one sentiment inference plus ABSA plus optional sarcasm logic in one request

- `/explain` is heavier than `/predict` because it constructs SHAP explanations per request
- `/batch_predict` skips ABSA to reduce cost, but still performs full sentiment inference and row reconstruction synchronously
- the backend remains CPU-sensitive because the compose file explicitly increases thread counts and CPU limits

14.2 Performance Risk Matrix

Area	Risk level	Evidence-backed reason
Startup time	High	model load and ABSA preload both occur before service readiness
Single prediction latency	Medium	one request can include language resolution, sentiment, ABSA, and sarcasm
Explain latency	High	SHAP explanation is generated on demand and not cached
Batch throughput	Medium	chunked inference exists, but request remains synchronous and process-local
Memory consumption	High	backend holds global model state and may also load ABSA and sarcasm ONNX sessions
Horizontal scalability	High	singleton in-process model and local memory state complicate concurrent scale-out semantics

15 Section 15 - Failure Mode Analysis

15.1 Failure Tree Overview

```

1 flowchart TD
2   StartupFail[Startup failure] --> ArtifactMissing[Missing model artifacts]
3   StartupFail --> ABSAFail[ABSA preload failure]
4   PredictFail[Prediction failure] --> UnsupportedLang[Unsupported language]
5   PredictFail --> RuntimeError[Inference runtime error]
6   ExplainFail[Explain failure] --> SHAPFail[SHAP pipeline failure]
7   BatchFail[Batch failure] --> CsvError[Invalid CSV]
8   BatchFail --> ThreadError[Batch inference exception]
9   EvalFail[Evaluation failure] --> DataMissing[Processed data missing]
10  EvalFail --> MlflowError[MLflow logging failure]
```

15.2 Failure Mode Table

Workflow	Symptoms	Probable causes	Affected components	Recovery path
Startup	API never becomes healthy	missing artifacts, bad model mode, ABSA preload error	backend, frontend readiness, monitoring scrape target	fix artifact/config issue and restart container

Workflow	Symptoms	Probable causes	Affected components	Recovery path
Prediction	400 or 500 response, frontend error bubble	unsupported language, model runtime exception, unloaded model	backend route, model layer, frontend chat flow	inspect model mode and artifacts; validate input language
Explanation	slow or failed explanation request	SHAP initialization, tokenizer/pipeline issues, resource pressure	explain route and message bubble UX	retry after model stabilization; consider disabling SHAP in weak environments
Batch processing	modal shows API error, no results returned	invalid CSV, missing text column, threaded inference failure	batch route and batch modal	fix input file or investigate model/runtime exception
Evaluation	status remains failed or last error populated	processed data missing, MLflow issue, evaluation exception	background task and MLflow integration	inspect <code>data/processed/sentences.csv</code> and MLflow URI
Training	adapter not produced	raw data missing, training config issue, trainer failure	DVC stages and scripts	rerun required upstream stages and verify params
Deployment	service starts partially	missing build secrets, DVC pull failure, compose misconfig	container stack	rebuild with valid secrets or pre-fetched artifacts
Monitoring	dashboards empty or stale	scrape target unavailable, misconfigured data source	Prometheus and Grafana	verify <code>/metrics</code> and provisioned configs

16 Section 16 - Technical Debt Analysis

16.1 Debt Register

Debt type	Severity	Evidence
Architectural debt	High	serving backend mixes HTTP, orchestration, and long-running evaluation control in one file and one process
Operational debt	High	evaluation state is in memory only, batch status is mocked, no durable job store
Security debt	Critical	no auth, wildcard CORS, default Grafana admin password, public admin-style routes
ML debt	Medium	artifact lifecycle and runtime mode selection are local-file based rather than registry-governed
Testing debt	Medium	tests exist broadly, but no evidence here of end-to-end container smoke tests or auth-related tests because auth is absent
Documentation debt	Medium	older docs still describe the system as a microservice architecture, while code shows a modular monolith
Frontend debt	Medium	large inline templates and UI logic concentration in <code>AppComponent</code> and batch component

17 Section 17 - Scalability Analysis

17.1 Application Scalability

The backend can scale only within the limits of in-process model serving. Horizontal scale is possible at the container level, but stateful assumptions remain weakly defined because evaluation state is local to one process and there is no shared queue or job store.

17.2 Model Scalability

- sentiment scoring scales better in ONNX mode than SHAP and ABSA
- sarcasm scoring adds per-text overhead
- SHAP does not scale well for high request concurrency in the current design

17.3 Operational Scalability

- local Compose is sufficient for development and demos
- there is no orchestration manifest for Kubernetes or rolling replacement
- there is no externalized queue, cache, or database for cross-instance coordination

17.4 Primary Bottlenecks

- startup model load time
- SHAP request cost
- ABSA request cost

- synchronous batch route
- single backend process owning mixed workloads

18 Section 18 - Architectural Assessment

18.1 Scorecard

Dimension	Score / 10	Justification
Architecture quality	7	coherent modular monolith with explicit online and offline planes, but service boundaries remain coarse
Maintainability	6	contracts and tests help, but backend and frontend central files are still too concentrated
Reliability	5	core inference path works, but startup heaviness, in-memory state, and mocked batch status reduce resilience
Security	3	no auth, permissive CORS, weak defaults, and public admin-style routes
Performance	6	ONNX path and batch chunking help, but SHAP and ABSA are expensive and colocated with serving
Observability	5	metrics and dashboards exist, but visibility remains shallow and logging is minimal
Extensibility	7	model interface, DVC stages, and script layout support extension reasonably well
Operational readiness	5	Compose and Dockerfiles exist, but rollout, secret handling, and durable job control are weak
Testing maturity	7	broad unit and module test surface across contracts, data, model, scripts, and training
ML lifecycle maturity	7	clear artifact path with DVC, MLflow, finetuning, export, and benchmarking

19 Section 19 - Recommendations

19.1 Immediate Improvements

Problem	Evidence	Priority	Complexity	Expected outcome
Unauthenticated public surface	all routes in <code>src/main.py</code> are public	Critical	Medium	basic production boundary control
Mocked batch status	<code>/batch_status</code> returns fixed payload	High	Medium	truthful batch semantics or explicit endpoint removal
In-memory evaluation state	<code>_evaluate_state</code> is process-local	High	Medium	durable multi-run observability
Weak dashboard security	Grafana password is <code>admin</code>	High	Low	reduce trivial support-service compromise

Problem	Evidence	Priority	Complexity	Expected outcome
Permissive CORS	wildcard origin with credentials	High	Low	stronger browser boundary hygiene

19.2 Short- to Long-term Roadmap

- Short term: move evaluation and batch jobs to durable job records; add upload size limits and route protection; emit richer metrics for batch, startup, SHAP, and evaluation.
- Medium term: split long-running and heavy analysis workloads from the request-serving API; introduce a proper config and secret strategy per environment.
- Long term: adopt model registry governance, environment-specific artifact promotion, and workload isolation for explanation and offline evaluation.

20 Section 20 - Final Technical Conclusion

The repository is a technically meaningful full-stack AI system rather than a partial demo. It contains a real user interface, a real FastAPI contract surface, a functional inference abstraction, a reproducible artifact pipeline, exportable ONNX serving assets, monitoring integration, and broad tests. Those are genuine strengths.

The decisive weaknesses are not missing architecture diagrams or missing concepts; they are missing production controls. The current implementation does not protect endpoints, does not persist long-running job state, uses a mocked batch-status endpoint, colocates heavy analysis with request serving, and exposes support services with weak defaults.

Final verdict: **strong implementation-first assessment target, suitable for learning, demonstration, and controlled internal use; not yet suitable for open production exposure without security, durability, and workload-isolation remediation.**